

STM32 Nucleo-F446RE

2축 짐벌 + 메카넘 4륜 + 라인 트레이서 개발 가이드

프로젝트	마이크로프로세서 실험 - 차량용 수평 유지 시스템
기간	20일 (Day 1 ~ Day 20)
MCU	STM32 Nucleo-F446RE (180MHz)
IDE	STM32CubeIDE 1.19.0 (HAL 라이브러리)
주요 부품	MPU6050, MG996R x2, BTS7960 x4, JGB37-520 x4
주요 부품 2	XL4015 벅 컨버터, LiPo 11.1V 2200mAh, TCRT5000 5채널

본 문서는 STM32 Nucleo-F446RE 기반 자율주행 짐벌 차량 프로젝트의 20일 단계별 개발 가이드와 모든 코드 모듈을 포함합니다. 각 단계는 작업 내용 + 검증 방법으로 구성되어 있으며, 코드는 그대로 STM32CubeIDE 프로젝트에 복사하여 사용 가능합니다.

목차

1. 전체 일정 개요 (20일)
2. 핀맵 및 결선 가이드
3. Day 1: 빵판 + GND/전원 레일 만들기
4. Day 2: Nucleo 단독 + 시리얼 통신
5. Day 3: MPU6050 추가
6. Day 4: 서보 1개 (M1) 추가
7. Day 5: 서보 2개 → 짐벌 완성
8. Day 6: XL4015 6V 조정 + 전원 분리
9. Day 7: LiPo 메인 전원 전환
10. Day 8-9: BTS7960 1개 + 모터 1개
11. Day 10-11: BTS7960 4개 + 모터 4개 (메카넘 8방향)
12. Day 12-13: 라인센서 5채널
13. Day 14: 라인 추종 PID
14. Day 15-16: 엔코더 1~4개
15. Day 17-20: 통합 + 시연 + 발표

[코드 부록] - 모든 모듈 소스

- A1. mpu6050.h / mpu6050.c
- A2. attitude.h / attitude.c
- A3. pid.h / pid.c
- A4. servo.h / servo.c
- A5. bts7960.h / bts7960.c
- A6. mecanum.h / mecanum.c
- A7. linesensor.h / linesensor.c
- A8. linefollow.h / linefollow.c
- A9. encoder.h / encoder.c
- A10. main.c (USER CODE 통합본)

1. 전체 일정 개요 (20일)

일자	작업	검증
Day 1	빵판 + GND/전원 레일 만들기	멀티미터로 단락 검사
Day 2	Nucleo 단독 + 시리얼 통신	부팅 메시지 출력
Day 3	MPU6050 추가	I2C 통신, 기울기 값
Day 4	서보 1개 (M1) 추가	손으로 흔들면 보정
Day 5	서보 2개 → 짐벌 완성	컵+물 흔들기 테스트
Day 6	XL4015 6V 조정 + 전원 분리	멀티미터로 6V 확인
Day 7	LiPo 메인 전원 전환	짐벌이 LiPo로도 동작
Day 8-9	BTS7960 1개 + 모터 1개 (FL)	단독 회전
Day 10-11	BTS7960 4개 + 모터 4개	메카넘 8방향
Day 12-13	라인센서 5채널	디버그 출력으로 확인
Day 14	라인 추종 PID	라인 따라 주행
Day 15-16	엔코더 1~4개	카운트 증가
Day 17-20	통합 + 시연 연습 + 발표 자료	최종 시연

△ 핵심 원칙: 한 단계 검증 전에는 절대 다음 단계로 가지 말 것.

2. 핀맵 및 결선 가이드

2.1 STM32 핀 ↔ Nucleo 보드 인쇄 이름

코드에서는 STM32 핀 이름(PA8, PB6 등)을 쓰지만 보드에는 Arduino 호환 이름(D7, A0 등)이 인쇄되어 있습니다. 점퍼선을 꽂을 때는 보드의 인쇄 글자를 보고 꽂으세요.

용도	코드 (STM32)	보드 인쇄	연결 대상
I2C SCL (MPU)	PB8	SCL/D15	MPU6050 SCL
I2C SDA (MPU)	PB9	SDA/D14	MPU6050 SDA
서보 M1 (Pitch)	PA8	D7	서보 M1 신호
서보 M2 (Roll)	PA9	D8	서보 M2 신호
모터 FL RPWM	PB6	PWM/CS/D10	BTS7960 FL RPWM
모터 FR RPWM	PC7	PWM/D9	BTS7960 FR RPWM
모터 RL RPWM	PB10	PWM/D6	BTS7960 RL RPWM
모터 RR RPWM	PB4	PWM/D5	BTS7960 RR RPWM
모터 FL LPWM	PB5	D4	BTS7960 FL LPWM
모터 FR LPWM	PB3	PWM/D3	BTS7960 FR LPWM
모터 RL LPWM	PA10	D2	BTS7960 RL LPWM
모터 RR LPWM	PA7	PWM/MOSI/D11	BTS7960 RR LPWM
라인센서 1	PA0	A0	라인센서 D1 (좌끝)
라인센서 2	PA1	A1	라인센서 D2
라인센서 3	PA4	A2	라인센서 D3 (중앙)
라인센서 4	PB0	A3	라인센서 D4
라인센서 5	PC1	A4	라인센서 D5 (우끝)
엔코더 FL	PC0	A5	FL 엔코더 A상
UART TX	PA2	TX/D1	(ST-LINK 내장)
UART RX	PA3	RX/D0	(ST-LINK 내장)

2.2 전원 결선

출처	전압	연결 대상
LiPo 11.1V (+)	11.1V	XL4015 IN+ / BTS7960 x4 VM+
XL4015 OUT	6.0V (조정 필수)	서보 M1, M2 빨강
Nucleo 3V3	3.3V	MPU6050 VCC
Nucleo 5V	5V	BTS7960 x4 VCC + R_EN + L_EN + 엔코더 + 라인센서
공통 GND	0V	모든 GND (LiPo-, Nucleo, MPU, 서보, BTS7960, 센서)

⚠ 절대 금지사항:

- 1) LiPo 11.1V를 Nucleo 5V 핀에 직결 (보드 손상)
- 2) XL4015 출력 조정 안 한 채로 서보 연결 (서보 손상)
- 3) BTS7960의 R_EN, L_EN 미연결 (모터 미작동)
- 4) 공통 GND 누락 (모터 노이즈로 짐벌 떨림, 시스템 리셋)

3. Day 1 — 빵판 + GND/전원 레일

목표: 전원 분배의 기반이 되는 공통 GND 망과 전원 레일을 만든다.

작업 내용

- 큰 빵판 1개 + 작은 빵판 1개 준비 (배터리/부품 분배용)
- 빵판 영역 매직펜으로 표시: 11.1V 레일, 공통 GND 레일, 5V 레일, 6V 레일
- 빵판 내부 GND 줄이 중간에 끊겨있는지 멀티미터 연속음으로 확인
- 두 빵판의 GND 줄을 점퍼선으로 서로 연결
- Nucleo CN6의 GND 핀 → 빵판 GND 레일 점퍼선 연결

검증 (체크리스트)

빵판 1의 GND 줄 양 끝에서 멀티미터 연속음 (삐 소리)

빵판 2의 GND 줄 양 끝에서 연속음

빵판 1 ↔ 빵판 2 GND 사이 연속음

Nucleo GND ↔ 빵판 GND 연속음

11.1V 레일과 GND 레일은 연속음 안 남 (단락 없음)

⚠ 멀티미터가 없으면 Day 2 단계에서 동작 확인하면서 검증 가능

⚠ 빵판 +/- 줄은 중간에서 끊겨있는 경우가 많음 (보드마다 다름)

4. Day 2 — Nucleo 단독 + 시리얼 통신

목표: STM32CubeIDE로 부팅 메시지를 시리얼로 출력하는 환경 구축.

작업 내용

- STM32CubeIDE 1.19.0 설치 (st.com 무료)
- File → New → STM32 Project → Board Selector → NUCLEO-F446RE 선택
- 프로젝트 이름 정하고 Finish → 'Initialize all peripherals?' → No
- .ioc 화면에서 RCC → HSE: Crystal/Ceramic Resonator
- .ioc 화면에서 SYS → Debug: Serial Wire
- USART2 → Mode: Asynchronous (이미 활성화돼있을 수 있음)
- Clock Configuration 탭 → HCLK = 180 입력 → Enter → Yes
- Ctrl+S로 코드 자동 생성
- main.c의 USER CODE 영역에 Hello World 코드 추가 (다음 페이지 참고)
- Ctrl+B로 빌드 → 0 errors
- USB 연결 → Debug (벌레 아이콘 또는 F11) → 자동 플래시
- Resume (F8) → 실행

- PuTTY 또는 Tera Term 실행 → ST-LINK COM 포트, 115200 baud로 연결

검증 (체크리스트)

시리얼에 '=== Hello Nucleo F446RE ===' 메시지 출력

0.5초마다 'Tick: 0, 1, 2...' 추가 출력

보드의 초록 LED (LD2)가 0.5초마다 깜빡임

⚠ 플래시 실패 시 USB 케이블 교체 + RESET 버튼(B2) 한 번 누르고 재시도

⚠ 시리얼 안 보이면 baudrate 115200, COM 포트 (장치 관리자에서 확인)

Day 2 코드 — main.c의 USER CODE 영역

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
/* USER CODE END Includes */

/* USER CODE BEGIN 0 */
#ifdef __GNUC__
int __io_putchar(int ch) {
    HAL_UART_Transmit(&huart2, (uint8_t*)&ch, 1, 10);
    return ch;
}
#endif
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
printf("\r\n=== Hello Nucleo F446RE ===\r\n");
uint32_t count = 0;
/* USER CODE END 2 */

/* (while 루프 안) */
/* USER CODE BEGIN 3 */
printf("Tick: %lu\r\n", count++);
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
HAL_Delay(500);
/* USER CODE END 3 */
```

5. Day 3 — MPU6050 추가

목표: I2C로 IMU 센서값을 읽고 시리얼로 출력 확인.

작업 내용

- MPU6050 결선 (전원 OFF 상태):
- MPU VCC → Nucleo 3V3 / MPU GND → 공통 GND
- MPU SCL → Nucleo SCL/D15 (PB8)
- MPU SDA → Nucleo SDA/D14 (PB9)
- MPU ADO → 공통 GND (주소 0x68)
- .ioc에서 PB8을 I2C1_SCL, PB9를 I2C1_SDA로 설정
- Connectivity → I2C1 → Speed Mode: Fast Mode (400kHz)
- Ctrl+S 코드 재생성
- 프로젝트의 Core/Inc/에 mpu6050.h 복사, Core/Src/에 mpu6050.c 복사
- main.c에 #include "mpu6050.h" 추가
- MPU6050_Init 호출 + 무한루프에서 MPU6050_ReadAll 호출
- 시리얼로 가속도/자이로 값 출력 코드 작성

검증 (체크리스트)

시리얼에 'MPU6050 OK' 메시지

Ax, Ay, Az, Gx, Gy, Gz 6개 값 출력

보드를 평평하게 두면 $Az \approx 1.0$, $Ax/Ay \approx 0.0$

보드를 기울이면 가속도 값이 따라 변함

정지 상태에서 자이로 ≈ 0 (작은 드리프트 가능)

⚠ 'MPU6050 init FAIL' → SCL/SDA 결선 확인, ADO 핀 GND 확인

⚠ 값이 전부 0 → I2C 통신 안 됨, 풀업 저항 (GY-521에 내장돼있음)

6. Day 4 — 서보 1개 (M1) 추가

목표: PWM 50Hz로 서보 1개의 각도 제어를 단독 테스트.

작업 내용

- 서보 M1만 단독으로 임시 결선 (XL4015 없이):
- 서보 빨강 → Nucleo 5V (임시, 짧은 테스트만!)
- 서보 갈색 → 공통 GND
- 서보 주황 (신호) → Nucleo D7 (PA8)
- .ioc에서 PA8 → TIM1_CH1 선택

- TIM1 설정: Prescaler 179, Counter Period 19999, Channel 1 PWM, Pulse 1500
- Ctrl+S 재생성
- servo.h, servo.c를 Core에 복사
- Servo_Init + Servo_StartAll 호출 후 -30 → 0 → +30도 반복 코드 작성
- ⚠ 서보를 짐벌 기구에 부착하지 말고 단독으로 테이블 위에서 테스트

검증 (체크리스트)

서보가 -30° → 0° → +30° 순서로 부드럽게 움직임
움직임에 떨림 없음
시리얼에 현재 각도 출력

⚠ 서보가 안 움직이면 TIM1 설정 (PSC=179, ARR=19999) 재확인

⚠ 서보 1개만! 2개 동시에 Nucleo 5V로 돌리면 전류 부족으로 보드 리셋

⚠ 테스트 후 서보 빨강을 5V에서 제거 (Day 6에서 XL4015 6V로 다시 연결)

7. Day 5 — 서보 2개 → 짐벌 완성

목표: 짐벌 기구 + IMU + PID로 수평 유지 자동 보정 구현.

작업 내용

- 짐벌 기구 조립 (베이스 + 좌우 기둥 + 외측 프레임 + 플레이트)
- 서보 M1을 왼쪽 기둥에 부착 (Pitch 축)
- 서보 M2를 외측 프레임 측면에 부착 (Roll 축)
- 조립 전 코드로 두 서보를 중립(0도)에 위치시킨 후 결합
- MPU6050을 플레이트 중앙에 양면테이프로 부착
- .ioc에서 PA9 → TIM1_CH2 추가, Channel 2 PWM, Pulse 1500
- TIM7 추가: Activated, Prescaler 179, Counter Period 4999 (200Hz)
- NVIC: TIM7 global interrupt 활성화, Priority 1
- attitude.h/c, pid.h/c, mpu6050 모두 통합
- main.c에 짐벌 통합 코드 적용 (다음 페이지 참고)
- PID 게인 튜닝: Kp 키워 진동 직전까지 → Kd 추가 → Ki 마지막
- 시작값: Kp=4.0, Ki=0.10, Kd=0.50

검증 (체크리스트)

부팅 시 'Calibrating gyro... DO NOT MOVE!' 메시지 동안 정지
캘리브레이션 후 'Gyro bias' 출력, 짐벌이 수평 자세 유지
베이스를 손으로 기울여도 플레이트는 수평 유지
플레이트 위 종이컵 + 물 1/3 올리고 베이스 흔들기 → 물 안 쏟아짐

시연 영상 촬영 (발표 자료용)

⚠ PID 발진 (떨림) → Kd 너무 큼, 작게 줄이기

⚠ 보정 느림 → Kp 증가

⚠ 정상상태 오차 (기울어진 채로 멈춤) → Ki 추가

⚠ 방향이 반대 (보드와 같이 기울어짐) → `SERVO_PITCH/ROLL_REVERSE = 1`

8. Day 6 — XL4015 6V 조정 + 전원 분리

목표: 서보 전용 6V 전원 라인을 만들고 Nucleo 5V 부담 줄이기.

작업 내용

- LiPo 분리 상태에서 XL4015 결선:
- LiPo 11.1V (+) → XL4015 IN+
- LiPo (-) → XL4015 IN- → 공통 GND
- LiPo 연결
- 멀티미터를 XL4015 OUT+ / OUT-에 대고 전압 측정
- XL4015의 파란 가변저항을 작은 드라이버로 돌려가며 출력 전압 조정
- 정확히 6.0V ($\pm 0.1V$) 맞추기
- LiPo 분리
- 서보 M1, M2의 빨강선을 Nucleo 5V에서 분리하고 XL4015 OUT+에 연결
- 두 서보의 갈색선은 공통 GND에 연결
- 다시 LiPo 연결 → USB도 연결 → 짐벌 동작 확인

검증 (체크리스트)

XL4015 출력이 정확히 6.0V (멀티미터 확인)

서보 2개 동시에 동작 (Day 4 때 1개만 됐던 거 → 이제 2개)

Nucleo 리셋 없이 안정적 동작

⚠ 6V 조정 안 한 채로 서보 연결하면 서보 손상! 반드시 측정 후 연결

⚠ XL4015는 부하 연결 전후 출력이 약간 다를 수 있음 → 부하 연결 후 재확인

9. Day 7 — LiPo 메인 전원 전환

목표: USB 케이블 없이 LiPo만으로 짐벌 동작 (시연 환경 시뮬레이션).

작업 내용

- 옵션 A (개발 중): USB로 Nucleo 전원, LiPo로 서보만 → 권장
- 옵션 B (시연 시): USB 빼고 Nucleo VIN 핀에 LiPo 11.1V 직결
- ⚠ Nucleo의 VIN은 7~12V 입력 가능 (5V 핀에 11.1V 직결 금지!)
- 옵션 B 결선:
- LiPo (+) → Nucleo VIN (CN6 맨 아래쪽)
- LiPo (-) → Nucleo GND
- XL4015도 동일 LiPo에서 분기
- USB 빼고 부팅 확인

- ⚠ 시리얼 디버그 출력은 USB 통해서 나오므로 LiPo만으로는 시리얼 못 봄

검증 (체크리스트)

USB 없이 LiPo만으로 부팅
 보드 LED가 켜짐
 짐벌이 수평 유지 동작 (USB 없이도)
 LiPo 전압 11.1V 이상 유지 (만충 12.6V)

- ⚠ 개발 중에는 USB로 디버그하는 게 편함
- ⚠ 시연 직전에 USB 빼고 LiPo만으로 동작 확인 필수
- ⚠ LiPo 만충 12.6V → 정상 11.1V → 방전 10.5V 이하면 위험

10. Day 8-9 — BTS7960 1개 + 모터 1개 (FL)

목표: 단일 H-브리지 모터 드라이버 결선 + 정/역 회전 테스트.

작업 내용

- FL 위치의 BTS7960 결선:
- VM+ → LiPo 11.1V (+) 레일
- VM- → 공통 GND
- VCC → Nucleo 5V (로직 전원)
- GND → 공통 GND
- R_EN, L_EN → Nucleo 5V (둘 다, 항상 활성화)
- RPWM → Nucleo PWM/CS/D10 (PB6)
- LPWM → Nucleo D4 (PB5)
- M+, M- → JGB37-520 모터의 굵은 빨강/검정 선
- ⚠ 모터는 차체에 부착하지 말고 테이블 위 단독으로 테스트
- .ioc에서 PB6 → TIM4_CH1, PB5 → GPIO_Output (또는 다른 TIM 채널)
- TIM4 설정: Prescaler 0, Counter Period 8999 (20kHz)
- Channel 1: PWM Generation
- bts7960.h/c 복사
- BTS7960_Init 호출 후 SetSpeed(0.3) → Stop → SetSpeed(-0.3) → Stop 반복

검증 (체크리스트)

모터가 정방향 회전 2초 → 정지 1초 → 역방향 2초 → 정지 → 반복
 회전 속도가 일정함 (떨림 X)
 전류 측정 시 무부하 0.5A 이내

⚠ 안 돌면: R_EN, L_EN 둘 다 5V에 연결됐는지 확인

⚠ VM+가 11.1V에 도달했는지 멀티미터 확인

⚠ 반대로 돌면: 모터 M+/M- 두 선 위치 바꾸기 또는 코드 REVERSE 플래그

⚠ 삐 소리만 나고 안 돌면: PWM 신호 안 가는 것 (핀 설정 확인)

11. Day 10-11 — BTS7960 4개 + 모터 4개 (메카넘 8방향)

목표: 메카넘 4륜 운동학으로 전후좌우 + 평행이동 + 회전 구현.

작업 내용

- 차량 프레임에 모터 4개 부착, 메카넘 휠 4개 X자 패턴 조립
- △ 위에서 봤을 때 4개 휠 롤러가 X자를 이뤄야 함 (틀리면 평행이동 X)
- FL, RR: 롤러가 '\\\ ' 방향
- FR, RL: 롤러가 '/// ' 방향
- 나머지 BTS7960 3개도 동일하게 결선:
- FR: RPWM→PWM/D9 (PC7), LPWM→PWM/D3 (PB3)
- RL: RPWM→PWM/D6 (PB10), LPWM→D2 (PA10)
- RR: RPWM→PWM/D5 (PB4), LPWM→PWM/MOSI/D11 (PA7)
- 각 BTS7960의 VM+, VM-, VCC, GND, R_EN, L_EN 모두 동일 결선
- .ioc 핀 설정 + 타이머 추가 (TIM8, TIM3 등)
- TIM6 추가: 100Hz (Prescaler 179, ARR 9999), NVIC 활성화 (차량 인터럽트)
- mecanum.h/c 복사, BTS7960 4개 인스턴스 생성
- UART 키보드 명령 처리 (w/a/s/d/q/e/space)

검증 (체크리스트)

- 'w' → 4륜 모두 정방향 → 전진
- 's' → 4륜 모두 역방향 → 후진
- 'a' → 좌측 평행이동 (차체 회전 없이 옆으로) 메카넘 핵심!
- 'd' → 우측 평행이동
- 'q' → 좌회전 (제자리)
- 'e' → 우회전 (제자리)
- 스페이스 → 정지
- 각 동작의 영상 촬영 (발표용)

- △ 한 바퀴 방향 반대 → 그 모터의 WHEEL_xx_REVERSE = 1
- △ 평행이동이 회전처럼 됨 → 메카넘 휠 X 패턴 조립 다시 확인
- △ 처음 MECANUM_MAX_OUT = 0.4 (안전), 동작 확인 후 0.6~0.8로 증가
- △ 차체가 무거우면 0.5 이하에서 못 움직임 → MAX_OUT 증가 + LiPo 충전 확인

12. Day 12-13 — 라인센서 5채널

목표: TCRT5000 5채널 모듈로 검정 라인 위치 추정.

작업 내용

- TCRT5000 5채널 라인 모듈 결선:
- VCC → Nucleo 5V
- GND → 공통 GND
- D1 (좌끝) → A0 (PA0)
- D2 → A1 (PA1)
- D3 (중앙) → A2 (PA4)
- D4 → A3 (PB0)
- D5 (우끝) → A4 (PC1)
- .ioc에서 5개 핀 → GPIO_Input + Pull-up
- linesensor.h/c 복사
- 디버그 코드로 raw 값 + position 출력
- 검정 절연테이프로 라인 트랙 만들기 (직선 + 곡선)
- 센서를 라인 위에서 좌우로 움직이며 값 변화 관찰

검증 (체크리스트)

흰 종이 위: Raw 0 0 0 0 0 → (LOST)

검정 라인 중앙: Raw 0 0 1 0 0 → Pos 0.00

라인을 왼쪽으로: Pos 음수 (-2.0에 가까워짐)

라인을 오른쪽으로: Pos 양수 (+2.0에 가까워짐)

⚠ raw 값이 반대 (검정 위에 1이 아니라 0) → linesensor.h의 LINE_ACTIVE_LOW 뒤집기

⚠ D1~D5 좌우 순서가 코드와 반대 → LineSensor_Init 인자 순서 바꾸기

⚠ 조명에 따라 임계값 흔들리면 → 발표장 조명 미리 확인 + 라인 테이프 무광 추천

13. Day 14 — 라인 추종 PID

목표: 라인 위치를 회전 명령으로 변환하여 자율주행 구현.

작업 내용

- linefollow.h/c 복사
- main.c에 시스템 모드 추가 (MODE_LINE_FOLLOW)
- UART 명령 '3' → 라인 추종 모드
- LineFollow_Init 호출 시 계인 입력:
- LINE_BASE_SPEED = 0.2 (느리게 시작)
- LINE_KP = 0.6, LINE_KI = 0.0, LINE_KD = 0.1
- 차량을 라인 위에 올리고 모드 3 진입

- Kp 튜닝: 너무 작으면 따라가지 못함, 너무 크면 좌우로 진동
- 직선 → 곡선 순서로 테스트
- 안정되면 BASE_SPEED를 0.3, 0.4로 점진 증가

검증 (체크리스트)

- 차량이 직선 라인을 따라 주행
- 곡선에서도 라인 이탈 없이 따라감
- 급격한 코너에서 멈추지 않음
- 라인 끝나면 line_lost 감지 후 정지
- 짐벌도 동시에 보정 동작 (모드 3은 짐벌 + 라인 추종)

- ⚠ 발진(좌우 진동) → Kp 감소 또는 Kd 증가
- ⚠ 라인 못 따라감 → Kp 증가 또는 BASE_SPEED 감소
- ⚠ 한 방향으로만 잘 도는 경우 → mecanum.c의 ω 부호 확인

14. Day 15-16 — 엔코더 1~4개 (선택)

목표: JGB37-520 엔코더로 주행 거리 측정 (PID는 옵션).

작업 내용

- 엔코더 결선 (FL부터):
- 엔코더 VCC (주황) → Nucleo 5V
- 엔코더 GND (갈색) → 공통 GND
- 엔코더 A상 (노랑) → Nucleo A5 (PC0)
- 엔코더 B상 (초록) → 미연결
- .ioc에서 PC0 → GPIO_EXTI0, Rising Edge
- NVIC: EXTI line 0 활성화
- encoder.h/c 복사
- main.c에 Encoder_OnInterrupt 콜백 등록
- 테스트: 모터 단독 회전시켜 카운트 증가 확인
- 추가 엔코더 3개도 동일 (시간 부족하면 1개만으로도 충분)

검증 (체크리스트)

- 차량 1초 전진 후 정지 → 엔코더 카운트가 약 200~500 증가
- 역회전 시 카운트 감소
- 디버그 출력에 Enc[xxxx xxxx xxxx xxxx] 값 표시

- ⚠ 엔코더 색깔은 제조사마다 다름 → 데이터시트 확인

⚠ 엔코더 1개만 달아도 발표용으로 충분 (주행 거리 측정 어필)

⚠ B상까지 사용하면 회전 방향도 정확히 판단 가능 (시간 남으면)

15. Day 17-20 — 통합 + 시연 연습 + 발표

목표: 최종 통합 시연 + 발표 자료 + 리허설.

작업 내용

- [Day 17] 통합 시연 시험
- 모드 3 (라인 추종 + 짐벌) 풀 시연
- 짐벌 위 컵 + 물 + 라인 주행 → 물 안 쏟아짐 확인
- 메인 시연 영상 촬영 (백업 영상도 따로)
- [Day 18] 발표 자료 1차 완성
- 슬라이드: 프로젝트 소개 / 시스템 블록도 / 부품 / 짐벌 메커니즘 /
- 상보필터 / PID 튜닝 결과 / 메카닉 운동학 / 라인 추종 /
- 다중 인터럽트 / 시연 영상
- 발표 시간 배분 + 예상 질문 정리
- [Day 19] 리허설
- 실제 발표 환경처럼 시연
- 슬라이드별 시간 측정 + 조정
- 시연 실패 백업 시나리오 점검
- [Day 20] 발표 전날 최종 점검
- LiPo 만충 (12.6V)
- XL4015 출력 6V 재확인
- 점퍼선 헐거움 검사 (잡아당기기)
- 핫멜트로 중요 결선부 고정 (선택)
- USB 백업: 발표 PPT + 시연 영상
- 컵 + 물 + 라인 종이 챙기기
- 일찍 자기

검증 (체크리스트)

라인 추종 + 짐벌 동시 동작 (메인 시연)
차가 가속/감속 시에도 짐벌 수평 유지
발표 시간 안에 슬라이드 다 발표 가능
시연 시 백업 영상 준비 완료

⚠ 발표 30분 전 도착, 시연 1회 연습

⚠ 시연 실패해도 영상으로 대체 가능하니 너무 긴장하지 말 것

⚠ 어필 키워드: 상보필터, PID, 메카닉 역운동학, 다중 인터럽트 우선순위

16. 발표/면접 어필 포인트

① 다중 인터럽트 우선순위 설계

짐벌(200Hz, TIM7) > 차량(100Hz, TIM6) > UART > EXTI 순으로 NVIC 우선순위 부여. 짐벌 제어가 절대 후순위로 밀리지 않도록 설계.

② 상보 필터(Complementary Filter)

가속도(저주파 신뢰) + 자이로(고주파 신뢰) 융합. $\alpha = 0.98$ 로 자이로 가중치 높임. 칼만 필터보다 연산량 적고 임베디드에 적합.

③ Anti-windup PID

적분항이 출력 한계 넘으면 누적 중단 (clamping). 외란 후 빠른 복귀 가능.

④ Derivative on Measurement

미분항을 오차 대신 측정값 기반으로 계산. 모드 전환 시 D-kick 없음.

⑤ 메카닉 역운동학 + 정규화

3축 명령(v_x, v_y, ω) $\rightarrow$ 4륜 속도 분배. 최대치 1.0 초과 시 비례 축소 (saturation 방지).

⑥ 모듈 분리 설계

9개 모듈로 분리 (mpu6050, attitude, pid, servo, bts7960, mecanum, linesensor, linefollow, encoder). 재사용성/디버깅 용이.

⑦ 결정론적 제어 주기

인터럽트로 플래그 set, main 루프에서 실제 처리. RTOS 없이도 정확한 5ms / 10ms 제어 주기 달성.

코드 부록 - 모든 모듈 소스

이 페이지부터 모든 .h / .c 파일의 전체 코드를 수록합니다. STM32CubeIDE 프로젝트의 Core/Inc/, Core/Src/ 폴더에 그대로 복사하여 사용 가능합니다. 각 모듈은 다음 순서로 수록됩니다:

- A1. mpu6050.h / mpu6050.c - MPU6050 I2C 통신
- A2. attitude.h / attitude.c - 상보 필터 자세 추정
- A3. pid.h / pid.c - PID 제어기 (Anti-windup)
- A4. servo.h / servo.c - MG996R 서보 제어
- A5. bts7960.h / bts7960.c - BTS7960 H-브리지
- A6. mecanum.h / mecanum.c - 메카넘 운동학
- A7. linesensor.h / linesensor.c - 5채널 라인 센서
- A8. linefollow.h / linefollow.c - 라인 추종 PID
- A9. encoder.h / encoder.c - 엔코더 EXTI 카운트
- A10. main.c USER CODE - 통합 메인 코드

A1. mpu6050

mpu6050.h

```
/**
 * @file mpu6050.h
 * @brief MPU6050 (GY-521) 드라이버 — STM32 HAL 기반
 * @author Gimbal Project
 *
 * I2C로 MPU6050에 접근하여 가속도/자이로 raw 데이터를 읽고,
 * 물리 단위(g, dps)로 변환한다.
 * 자세 추정(상보 필터)은 attitude.c 에서 별도로 수행한다.
 */
#ifndef __MPU6050_H
#define __MPU6050_H

#include "stm32f4xx_hal.h"
#include <stdint.h>
#include <stdbool.h>

/* MPU6050 I2C 주소 */
/* ADO = GND → 7-bit addr 0x68, HAL은 8-bit 받으므로 << 1 */
#define MPU6050_ADDR (0x68 << 1)

/* 레지스터 주소 */
#define MPU6050_REG_SMPLRT_DIV 0x19 // 샘플레이트 분주
#define MPU6050_REG_CONFIG 0x1A // DLPF 설정
#define MPU6050_REG_GYRO_CONFIG 0x1B // 자이로 풀스케일
#define MPU6050_REG_ACCEL_CONFIG 0x1C // 가속도 풀스케일
#define MPU6050_REG_ACCEL_XOUT_H 0x3B // 가속도/자이로 14바이트 burst 시작
#define MPU6050_REG_PWR_MGMT_1 0x6B // 전원 관리 (sleep wake)
#define MPU6050_REG_WHO_AM_I 0x75 // ID 확인 (0x68 반환)

/* 변환 스케일 */
/* 본 프로젝트 기본 설정:
 * - 가속도 풀스케일: ±2g → 16384 LSB/g
 * - 자이로 풀스케일: ±500dps → 65.5 LSB/dps
 * (서보 짐벌 회전 속도는 빠르지 않으므로 ±500dps면 충분)
 */
#define ACCEL_LSB_PER_G 16384.0f
#define GYRO_LSB_PER_DPS 65.5f

/* 데이터 구조체 */
typedef struct {
    /* Raw 값 (보정 전) */
    int16_t accel_raw[3]; // [0]=X, [1]=Y, [2]=Z
    int16_t gyro_raw[3];
    int16_t temp_raw;

    /* 변환된 SI 값 */
    float accel_g[3]; // 단위: g
    float gyro_dps[3]; // 단위: degree per second
    float temp_c; // 섭씨

    /* 자이로 바이어스 (정지 상태 보정값) */
    float gyro_bias[3];
} MPU6050_t;

/* 함수 프로토타입 */

/**
 * @brief MPU6050 초기화
 * @return HAL_OK / HAL_ERROR
 *
 * - WHO_AM_I 확인
 * - sleep 해제 (PWR_MGMT_1 = 0)
 * - 샘플레이트 1kHz (SMPLRT_DIV = 0)
 * - DLPF Bandwidth ~44Hz (CONFIG = 3)
 * - 자이로 ±500dps (GYRO_CONFIG = 0x08)
 * - 가속도 ±2g (ACCEL_CONFIG = 0x00)
 */
HAL_StatusTypeDef MPU6050_Init(I2C_HandleTypeDef *hi2c);

/**
 * @brief 가속도/자이로 raw 14바이트를 burst read 후 SI 단위로 변환
 * gyro_bias 가 미리 calibrate 되어있다면 자동 감산
 */
HAL_StatusTypeDef MPU6050_ReadAll(I2C_HandleTypeDef *hi2c, MPU6050_t *dev);

/**
```

```
* @brief 자이로 바이어스 calibration
*      센서를 평평한 곳에 정지시킨 상태에서 호출
*      500 샘플 평균을 bias 로 저장
* @note 호출 동안 약 1~2초 동안 절대 움직이지 말 것
*/
HAL_StatusTypeDef MPU6050_CalibrateGyro(I2C_HandleTypeDef *hi2c, MPU6050_t *dev);

#endif /* __MPU6050_H */
```

mpu6050.c

```
/**
 * @file mpu6050.c
 * @brief MPU6050 드라이버 구현
 */
#include "mpu6050.h"
#include <string.h>

#define I2C_TIMEOUT 100 // ms

/* 정적 헬퍼 함수: 1바이트 쓰기 */
static HAL_StatusTypeDef mpu_write_byte(I2C_HandleTypeDef *hi2c,
                                         uint8_t reg, uint8_t val)
{
    return HAL_I2C_Mem_Write(hi2c, MPU6050_ADDR, reg, 1, &val, 1, I2C_TIMEOUT);
}

/* 정적 헬퍼 함수: 다중 바이트 읽기 */
static HAL_StatusTypeDef mpu_read(I2C_HandleTypeDef *hi2c,
                                   uint8_t reg, uint8_t *buf, uint16_t len)
{
    return HAL_I2C_Mem_Read(hi2c, MPU6050_ADDR, reg, 1, buf, len, I2C_TIMEOUT);
}

/*
 * MPU6050_Init
 */
HAL_StatusTypeDef MPU6050_Init(I2C_HandleTypeDef *hi2c)
{
    uint8_t who = 0;
    HAL_StatusTypeDef st;

    /* 1) WHO_AM_I 확인 → 0x68 반환되어야 정상
     * ADO 핀이 HIGH면 0x69 반환되니까 결선 확인 필수 */
    st = mpu_read(hi2c, MPU6050_REG_WHO_AM_I, &who, 1);
    if (st != HAL_OK) return st;
    if (who != 0x68) return HAL_ERROR; // 센서 응답 이상

    /* 2) PWR_MGMT_1 = 0x00 → sleep 해제, 내부 8MHz 클럭 사용 */
    st = mpu_write_byte(hi2c, MPU6050_REG_PWR_MGMT_1, 0x00);
    if (st != HAL_OK) return st;
    HAL_Delay(50); // wake up 안정화

    /* 3) SMPLRT_DIV = 0 → sample rate = 1kHz (DLPF 활성화 가정시)
     * 실제로는 우리가 200Hz로 메인 루프에서 읽음 */
    st = mpu_write_byte(hi2c, MPU6050_REG_SMPLRT_DIV, 0x00);
    if (st != HAL_OK) return st;

    /* 4) CONFIG = 0x03 → DLPF Bandwidth ≈ 44Hz (acc), 42Hz (gyro)
     * 노이즈 감소 효과 큼. 너무 낮추면 응답 느려지고, 높이면 노이즈 ↑ */
    st = mpu_write_byte(hi2c, MPU6050_REG_CONFIG, 0x03);
    if (st != HAL_OK) return st;

    /* 5) GYRO_CONFIG = 0x08 → 자이로 풀스케일 ±500 dps */
    st = mpu_write_byte(hi2c, MPU6050_REG_GYRO_CONFIG, 0x08);
    if (st != HAL_OK) return st;

    /* 6) ACCEL_CONFIG = 0x00 → 가속도 풀스케일 ±2g */
    st = mpu_write_byte(hi2c, MPU6050_REG_ACCEL_CONFIG, 0x00);
    if (st != HAL_OK) return st;

    return HAL_OK;
}

/*
 * MPU6050_ReadAll
 */
HAL_StatusTypeDef MPU6050_ReadAll(I2C_HandleTypeDef *hi2c, MPU6050_t *dev)
{
    uint8_t buf[14];
    HAL_StatusTypeDef st;

    /* 14 바이트 burst read (가속도 6, 온도 2, 자이로 6) */
    st = mpu_read(hi2c, MPU6050_REG_ACCEL_XOUT_H, buf, 14);
    if (st != HAL_OK) return st;

    /* Big-endian 정렬: H 바이트가 먼저 */
    dev->accel_raw[0] = (int16_t)((buf[0] << 8) | buf[1]);
    dev->accel_raw[1] = (int16_t)((buf[2] << 8) | buf[3]);
    dev->accel_raw[2] = (int16_t)((buf[4] << 8) | buf[5]);
    dev->temp_raw = (int16_t)((buf[6] << 8) | buf[7]);
}
```

```

dev->gyro_raw[0] = (int16_t)((buf[8] << 8) | buf[9]);
dev->gyro_raw[1] = (int16_t)((buf[10] << 8) | buf[11]);
dev->gyro_raw[2] = (int16_t)((buf[12] << 8) | buf[13]);

/* SI 단위로 변환 */
for (int i = 0; i < 3; i++) {
    dev->accel_g[i] = (float)dev->accel_raw[i] / ACCEL_LSB_PER_G;
    /* 자이로는 바이어스 빼고 dps로 변환 */
    dev->gyro_dps[i] = ((float)dev->gyro_raw[i] / GYRO_LSB_PER_DPS)
        - dev->gyro_bias[i];
}
dev->temp_c = (float)dev->temp_raw / 340.0f + 36.53f; // 데이터시트 공식

return HAL_OK;
}

/*
 * MPU6050_CalibrateGyro
 * 정지 상태에서 500 샘플 평균 → bias 저장
 */
HAL_StatusTypeDef MPU6050_CalibrateGyro(I2C_HandleTypeDef *hi2c, MPU6050_t *dev)
{
    const int N = 500;
    float sum[3] = {0.0f, 0.0f, 0.0f};

    /* bias 0으로 초기화 후 raw 측정 */
    dev->gyro_bias[0] = 0.0f;
    dev->gyro_bias[1] = 0.0f;
    dev->gyro_bias[2] = 0.0f;

    for (int n = 0; n < N; n++) {
        if (MPU6050_ReadAll(hi2c, dev) != HAL_OK) return HAL_ERROR;
        sum[0] += dev->gyro_dps[0];
        sum[1] += dev->gyro_dps[1];
        sum[2] += dev->gyro_dps[2];
        HAL_Delay(2); // ≈ 500 Hz 샘플링
    }

    dev->gyro_bias[0] = sum[0] / N;
    dev->gyro_bias[1] = sum[1] / N;
    dev->gyro_bias[2] = sum[2] / N;

    return HAL_OK;
}

```

A2. attitude

attitude.h

```
/**
 * @file attitude.h
 * @brief 상보 필터 기반 자세 추정 (Pitch / Roll)
 *
 * - 가속도계: 정적일 때 정확한 각도, 동적일 때 노이즈 큼
 * - 자이로: 단기 응답 빠름, 장기 드리프트
 * → 상보 필터 (high-pass on gyro, low-pass on accel) 로 융합
 *
 *  $angle = \alpha * (angle + gyro * dt) + (1 - \alpha) * accel\_angle$ 
 *
 *  $\alpha$  는 보통 0.96 ~ 0.99 (gyro 가중치)
 * 짐벌처럼 정적 환경 위주면 0.96, 동적이면 0.98 ~ 0.99
 */
#ifndef __ATTITUDE_H
#define __ATTITUDE_H

#include "mpu6050.h"

typedef struct {
    float pitch;    // 도 단위, 외측 프레임이 보정해야 할 축
    float roll;     // 도 단위, 플레이트가 보정해야 할 축
    float alpha;    // 상보 필터 계수 (0~1, gyro 가중치)
    float dt;       // 샘플 주기 (초)
} Attitude_t;

/**
 * @brief 자세 추정기 초기화
 * 가속도 값으로 초기 각도 세팅
 */
void Attitude_Init(Attitude_t *att, MPU6050_t *dev, float alpha, float dt_sec);

/**
 * @brief 한 스텝 업데이트
 * 루프 주기마다 호출 (예: 5ms = 200Hz)
 */
void Attitude_Update(Attitude_t *att, MPU6050_t *dev);

#endif
```

attitude.c

```
/**
 * @file attitude.c
 * @brief 상보 필터 구현
 */
#include "attitude.h"
#include <math.h>

#define RAD_TO_DEG (57.2957795f)

/* 가속도계로부터 Pitch/Roll 계산 (단위: 도)
 *
 * 좌표계 정의 (MPU6050 PCB 기준, 일반적 셋업):
 * X축: 보드 길이 방향
 * Y축: 보드 폭 방향
 * Z축: 보드 위쪽 (중력 반대)
 *
 *  $Pitch = atan2(-Ax, \sqrt{Ay^2 + Az^2})$  [X축이 들리면 +]
 *  $Roll = atan2(Ay, Az)$  [Y축이 들리면 +]
 *
 * △ 센서를 플레이트에 부착하는 방향에 따라 부호가 뒤집힐 수 있음.
 * 짐벌이 잘못된 방향으로 움직이면 부호만 바꿔주면 됨.
 */
static void accel_to_angles(MPU6050_t *dev, float *pitch_deg, float *roll_deg)
{
    float ax = dev->accel_g[0];
    float ay = dev->accel_g[1];
    float az = dev->accel_g[2];

    *pitch_deg = atan2f(-ax, sqrtf(ay * ay + az * az)) * RAD_TO_DEG;
    *roll_deg = atan2f(ay, az) * RAD_TO_DEG;
}

/*
```

```

/* Attitude_Init
*/
void Attitude_Init(Attitude_t *att, MPU6050_t *dev, float alpha, float dt_sec)
{
    att->alpha = alpha;
    att->dt     = dt_sec;
    accel_to_angles(dev, &att->pitch, &att->roll); // 가속도로 초기값 세팅
}

/*
* Attitude_Update
*/
void Attitude_Update(Attitude_t *att, MPU6050_t *dev)
{
    /* 1) 가속도계로 즉시 추정된 Pitch/Roll */
    float pitch_acc, roll_acc;
    accel_to_angles(dev, &pitch_acc, &roll_acc);

    /* 2) 자이로 적분: 각속도(dps) × dt
    *
    *   Pitch 축: Y 자이로 (보드의 짧은 축 회전이 Pitch 변화)
    *   Roll  축: X 자이로
    *
    *   △ 짐벌의 회전 방향과 자이로 부호가 안 맞으면 부호 뒤집기
    */
    float gyro_pitch = dev->gyro_dps[1]; // Y
    float gyro_roll  = dev->gyro_dps[0]; // X

    float pitch_gyro = att->pitch + gyro_pitch * att->dt;
    float roll_gyro  = att->roll  + gyro_roll  * att->dt;

    /* 3) 상보 필터로 융합 */
    att->pitch = att->alpha * pitch_gyro + (1.0f - att->alpha) * pitch_acc;
    att->roll  = att->alpha * roll_gyro  + (1.0f - att->alpha) * roll_acc;
}

```

A3. pid

pid.h

```
/**
 * @file pid.h
 * @brief PID 제어기 (Anti-windup 포함)
 *
 * 짐벌은 setpoint = 0 (수평) 고정.
 * 제어 입력 u = 서보 각도 명령 (도)
 *
 *  $error = setpoint - measured$ 
 *  $u = K_p * e + K_i * \int e \, dt + K_d * de/dt$ 
 *
 * Anti-windup: 적분항이 출력 한계를 넘으면 적분 누적 중단 (clamping)
 * 미분 noise 완화: D term은 측정값 기반 (derivative on measurement)
 */
#ifndef __PID_H
#define __PID_H

typedef struct {
    /* 게인 */
    float Kp;
    float Ki;
    float Kd;

    /* 내부 상태 */
    float integral; // 적분 누적값
    float prev_measurement; // D term 용 (derivative on measurement)
    float prev_error; // 디버깅/대안용

    /* 한계 */
    float out_min; // 출력 하한 (도)
    float out_max; // 출력 상한 (도)
    float integral_max; // 적분 누적 한계 (anti-windup)

    /* 샘플 주기 */
    float dt;
} PID_t;

void PID_Init(PID_t *pid, float Kp, float Ki, float Kd,
              float out_min, float out_max,
              float integral_max, float dt_sec);

/**
 * @brief PID 한 스텝 계산
 * @param setpoint 목표값 (도). 짐벌은 보통 0
 * @param measurement 현재 측정값 (도)
 * @return 제어 출력 (도) — out_min ~ out_max 로 클램프됨
 */
float PID_Compute(PID_t *pid, float setpoint, float measurement);

/**
 * @brief 내부 상태 초기화 (시동 시 큰 적분 튀는 것 방지)
 */
void PID_Reset(PID_t *pid);

#endif
```

pid.c

```
/**
 * @file pid.c
 * @brief PID 구현 — Anti-windup + Derivative on Measurement
 */
#include "pid.h"

/* _____ */
void PID_Init(PID_t *pid, float Kp, float Ki, float Kd,
              float out_min, float out_max,
              float integral_max, float dt_sec)
{
    pid->Kp = Kp;
    pid->Ki = Ki;
    pid->Kd = Kd;
    pid->out_min = out_min;
    pid->out_max = out_max;
```

```

    pid->integral_max = integral_max;
    pid->dt = dt_sec;
    PID_Reset(pid);
}

/* ----- */
void PID_Reset(PID_t *pid)
{
    pid->integral = 0.0f;
    pid->prev_measurement = 0.0f;
    pid->prev_error = 0.0f;
}

/* ----- */
float PID_Compute(PID_t *pid, float setpoint, float measurement)
{
    /* 1) 오차 계산 */
    float error = setpoint - measurement;

    /* 2) P term */
    float P_out = pid->Kp * error;

    /* 3) I term — Anti-windup (clamping)
     *   적분이 한계 넘으면 saturation, 출력이 한계 풀리면 자연 해제 */
    pid->integral += error * pid->dt;
    if (pid->integral > pid->integral_max) pid->integral = pid->integral_max;
    else if (pid->integral < -pid->integral_max) pid->integral = -pid->integral_max;
    float I_out = pid->Ki * pid->integral;

    /* 4) D term — Derivative on Measurement (노이즈 완화)
     *   de/dt = -d(meas)/dt (setpoint 가 갑자기 바뀌어도 D kick 없음) */
    float d_meas = (measurement - pid->prev_measurement) / pid->dt;
    float D_out = -pid->Kd * d_meas;

    pid->prev_measurement = measurement;
    pid->prev_error = error;

    /* 5) 합산 + 출력 클램프 */
    float u = P_out + I_out + D_out;
    if (u > pid->out_max) u = pid->out_max;
    else if (u < pid->out_min) u = pid->out_min;

    return u;
}

```

A4. servo

servo.h

```
/**
 * @file    servo.h
 * @brief   MG996R 서보 제어 (PWM 50Hz)
 *
 * Timer 설정: prescaler=179, ARR=19999 (180MHz → 1tick=1μs, 주기=20ms)
 * Pulse 값(=CCR)을 μs 단위로 그대로 쓰면 됨.
 *
 * 1000 μs → -90도
 * 1500 μs → 0도 (중립)
 * 2000 μs → +90도
 *
 * MG996R 실측은 모듈마다 약간 다를 수 있어 SERVO_MIN/MAX_US 로 보정.
 */
#ifndef __SERVO_H
#define __SERVO_H

#include "stm32f4xx_hal.h"
#include <stdint.h>

/* ----- 펄스 폭 한계 ----- */
* 실측 결과에 따라 미세조정 가능.
* MG996R 일반 사양: 1000~2000μs (180도 범위)
* 안전마진: 900~2100 정도까지는 허용
*/
#define SERVO_MIN_US    1000    // -90도
#define SERVO_MID_US    1500    // 0도
#define SERVO_MAX_US    2000    // +90도

/* 짐벌 기계적 회전 한계 (소프트웨어 리미터)
 * 외측 프레임과 플레이트가 서로 충돌하지 않는 안전 각도
 * 처음 동작 시켜보고 좁히거나 넓히기
 */
#define GIMBAL_ANGLE_LIMIT_DEG 45.0f

typedef struct {
    TIM_HandleTypeDef *htim;    // 사용 타이머 핸들
    uint32_t channel;          // TIM_CHANNEL_1 또는 TIM_CHANNEL_2
    float offset_deg;          // 중립 위치 오프셋 (기계적 캘리브레이션용)
    int reverse;               // 1이면 부호 반전 (서보 방향이 반대일 때)
} Servo_t;

/**
 * @brief 서보 초기화 및 중립 위치로 이동
 */
void Servo_Init(Servo_t *s, TIM_HandleTypeDef *htim, uint32_t channel,
                float offset_deg, int reverse);

/**
 * @brief 각도(도)를 받아서 서보를 그 위치로 보냄
 * 내부에서 [-LIMIT, +LIMIT] 클램프 + offset/reverse 처리
 */
void Servo_WriteAngle(Servo_t *s, float angle_deg);

/**
 * @brief 모든 서보 PWM 출력 시작 (TIM_PWM_Start)
 * Servo_Init 후 한번 호출
 */
void Servo_StartAll(TIM_HandleTypeDef *htim);

#endif
```

servo.c

```
/**
 * @file    servo.c
 * @brief   MG996R PWM 제어 구현
 */
#include "servo.h"

/* 각도 [-90, +90] → 펄스 폭 [SERVO_MIN_US, SERVO_MAX_US] 선형 매핑 */
static uint32_t angle_to_us(float angle_deg)
{
    /* -90 ~ +90 클램프 */
}
```

```

    if (angle_deg < -90.0f) angle_deg = -90.0f;
    if (angle_deg > +90.0f) angle_deg = +90.0f;

    /* (angle + 90) / 180 * (max - min) + min */
    float us = ((angle_deg + 90.0f) / 180.0f) * (SERVO_MAX_US - SERVO_MIN_US)
        + SERVO_MIN_US;
    return (uint32_t)(us + 0.5f);
}

/* ----- */
void Servo_Init(Servo_t *s, TIM_HandleTypeDef *htim, uint32_t channel,
    float offset_deg, int reverse)
{
    s->htim      = htim;
    s->channel    = channel;
    s->offset_deg = offset_deg;
    s->reverse    = reverse;

    /* 시작은 중립 위치 */
    __HAL_TIM_SET_COMPARE(htim, channel, SERVO_MID_US);
}

/* ----- */
void Servo_WriteAngle(Servo_t *s, float angle_deg)
{
    /* 1) 기계적 짐벌 한계 클램프 */
    if (angle_deg > GIMBAL_ANGLE_LIMIT_DEG) angle_deg = GIMBAL_ANGLE_LIMIT_DEG;
    if (angle_deg < -GIMBAL_ANGLE_LIMIT_DEG) angle_deg = -GIMBAL_ANGLE_LIMIT_DEG;

    /* 2) 부호 반전 (서보 방향이 반대일 때) */
    if (s->reverse) angle_deg = -angle_deg;

    /* 3) 오프셋 보정 (기계적 중립이 정확히 안 맞을 때) */
    angle_deg += s->offset_deg;

    /* 4) 펄스 폭 계산 후 CCR 갱신 */
    uint32_t us = angle_to_us(angle_deg);
    __HAL_TIM_SET_COMPARE(s->htim, s->channel, us);
}

/* ----- */
void Servo_StartAll(TIM_HandleTypeDef *htim)
{
    HAL_TIM_PWM_Start(htim, TIM_CHANNEL_1);
    HAL_TIM_PWM_Start(htim, TIM_CHANNEL_2);
}

```

A5. bts7960

bts7960.h

```
/**
 * @file    bts7960.h
 * @brief   BTS7960 H-브리지 모터 드라이버 — 단일 모터 제어
 *
 * BTS7960 모듈 1개 = 모터 1개 양방향 제어
 *   - RPWM (우방향 PWM): 정방향 회전 시 PWM
 *   - LPWM (좌방향 PWM): 역방향 회전 시 PWM
 *   - R_EN, L_EN: enable. 이 프로젝트에서는 5V 직결로 항상 활성화
 *
 * 한쪽만 PWM, 다른 쪽은 0으로 고정 (둘 다 PWM이면 쇼트 위험)
 *   정회전:  RPWM = duty, LPWM = 0
 *   역회전:  RPWM = 0,    LPWM = duty
 *   브레이크: RPWM = 0,    LPWM = 0 (또는 둘 다 HIGH = 코스트)
 *
 * PWM 주파수: 1~20kHz 권장 (모터 소음 vs 효율 절충, 본 코드 20kHz)
 */
#ifndef __BTS7960_H
#define __BTS7960_H

#include "stm32f4xx_hal.h"
#include <stdint.h>

typedef struct {
    TIM_HandleTypeDef *htim_rpwm;
    uint32_t          ch_rpwm;
    TIM_HandleTypeDef *htim_lpwm;
    uint32_t          ch_lpwm;
    uint32_t          arr;    // 타이머 ARR (PWM 최댓값)
    int               reverse; // 1이면 회전 방향 반전
} BTS7960_t;

/**
 * @brief 모터 초기화 (PWM 시작 포함)
 *
 * Init 후 자동으로 정지 상태
 */
void BTS7960_Init(BTS7960_t *m,
                  TIM_HandleTypeDef *htim_rpwm, uint32_t ch_rpwm,
                  TIM_HandleTypeDef *htim_lpwm, uint32_t ch_lpwm,
                  uint32_t arr, int reverse);

/**
 * @brief 모터 속도 설정
 * @param speed -1.0 ~ +1.0 (음수 역회전, 0 정지, +1.0 최대 정회전)
 *
 * 범위 벗어나면 자동 클램프
 */
void BTS7960_SetSpeed(BTS7960_t *m, float speed);

/** @brief 즉시 정지 (소프트 브레이크) */
void BTS7960_Stop(BTS7960_t *m);

#endif
```

bts7960.c

```
/**
 * @file    bts7960.c
 */
#include "bts7960.h"

void BTS7960_Init(BTS7960_t *m,
                  TIM_HandleTypeDef *htim_rpwm, uint32_t ch_rpwm,
                  TIM_HandleTypeDef *htim_lpwm, uint32_t ch_lpwm,
                  uint32_t arr, int reverse)
{
    m->htim_rpwm = htim_rpwm;
    m->ch_rpwm   = ch_rpwm;
    m->htim_lpwm = htim_lpwm;
    m->ch_lpwm   = ch_lpwm;
    m->arr       = arr;
    m->reverse   = reverse;

    /* PWM 시작 (CCR=0 이므로 무회전) */
    __HAL_TIM_SET_COMPARE(htim_rpwm, ch_rpwm, 0);
```

```

    __HAL_TIM_SET_COMPARE(htim_lpwm, ch_lpwm, 0);
    HAL_TIM_PWM_Start(htim_rpwm, ch_rpwm);
    HAL_TIM_PWM_Start(htim_lpwm, ch_lpwm);
}

void BTS7960_SetSpeed(BTS7960_t *m, float speed)
{
    /* 1) 클램프 */
    if (speed > 1.0f) speed = 1.0f;
    if (speed < -1.0f) speed = -1.0f;

    /* 2) 방향 반전 (조립 시 모터 +선/-선이 반대로 됐을 때 보정용) */
    if (m->reverse) speed = -speed;

    /* 3) duty 계산 */
    uint32_t duty = (uint32_t)(m->arr * (speed >= 0 ? speed : -speed));

    /* 4) 방향에 맞춰 R/L PWM 분배 (절대 둘 다 동시에 PWM 주지 말 것) */
    if (speed > 0.0f) {
        __HAL_TIM_SET_COMPARE(m->htim_rpwm, m->ch_rpwm, duty);
        __HAL_TIM_SET_COMPARE(m->htim_lpwm, m->ch_lpwm, 0);
    } else if (speed < 0.0f) {
        __HAL_TIM_SET_COMPARE(m->htim_rpwm, m->ch_rpwm, 0);
        __HAL_TIM_SET_COMPARE(m->htim_lpwm, m->ch_lpwm, duty);
    } else {
        /* speed == 0 → 정지 */
        __HAL_TIM_SET_COMPARE(m->htim_rpwm, m->ch_rpwm, 0);
        __HAL_TIM_SET_COMPARE(m->htim_lpwm, m->ch_lpwm, 0);
    }
}

void BTS7960_Stop(BTS7960_t *m)
{
    BTS7960_SetSpeed(m, 0.0f);
}

```

A6. mecanum

mecanum.h

```
/**
 * @file    mecanum.h
 * @brief   메카닉 4륜 운동학 (Inverse Kinematics)
 *
 * 3축 명령 (전진 vx, 횡이동 vy, 회전  $\omega$ ) → 4륜 속도
 *
 * FL = vx - vy -  $\omega$     (앞좌)
 * FR = vx + vy +  $\omega$     (앞우)
 * RL = vx + vy -  $\omega$     (뒤좌)
 * RR = vx - vy +  $\omega$     (뒤우)
 *
 * ※ Lx, Ly (휠베이스/트랙)는 본 단순화 버전에서  $\omega$  에 묶여 처리
 *
 * 입력 범위: 각 축 -1.0 ~ +1.0
 * 출력 범위: 4륜 속도 합이 1.0 넘으면 자동 정규화 (saturation 방지)
 *
 * 좌표계 (위에서 봤을 때):
 *      ↑ +vx (전진)
 *      |
 * FL ————— FR
 *      |   ↓+ $\omega$    |   ← + $\omega$  = 좌회전 (반시계, 위에서 본 기준)
 *      | → +vy      |
 * RL ————— RR
 *
 * △ 메카닉 휠 롤러 방향:
 * FL, RR : "\\" 방향 롤러
 * FR, RL : "/"  방향 롤러
 * 위에서 봤을 때 4개가 X자 패턴. 잘못 조립하면 횡이동 안 됨.
 */
#ifndef __MECANUM_H
#define __MECANUM_H

#include "bts7960.h"

typedef enum {
    WHEEL_FL = 0, // Front Left
    WHEEL_FR = 1, // Front Right
    WHEEL_RL = 2, // Rear Left
    WHEEL_RR = 3, // Rear Right
    WHEEL_COUNT = 4
} WheelIndex_t;

typedef struct {
    BTS7960_t *wheels[WHEEL_COUNT]; // 4륜 모터 핸들 (포인터)
    float      max_output;           // 전체 최대 출력 한계 (0.0~1.0)
} Mecanum_t;

/**
 * @brief 메카닉 시스템 초기화
 * @param fl, fr, rl, rr 미리 BTS7960_Init 된 모터 핸들 4개
 * @param max_output   안전 최대치 (0.4~0.7 추천, 처음엔 작게 시작)
 */
void Mecanum_Init(Mecanum_t *m,
                  BTS7960_t *fl, BTS7960_t *fr,
                  BTS7960_t *rl, BTS7960_t *rr,
                  float max_output);

/**
 * @brief 3축 명령 → 4륜 속도 분배 + 모터 출력
 * @param vx 전진/후진 [-1.0, +1.0] (+ 전진)
 * @param vy 좌/우 횡이동 [-1.0, +1.0] (+ 오른쪽)
 * @param wz 회전 각속도 [-1.0, +1.0] (+ 좌회전 = 반시계)
 *
 * 세 입력의 합 절댓값이 1.0 넘으면 비례 축소(정규화)
 */
void Mecanum_Drive(Mecanum_t *m, float vx, float vy, float wz);

/** @brief 즉시 4륜 정지 */
void Mecanum_Stop(Mecanum_t *m);

/* ————— 편의 함수 (자주 쓸 동작 미리 정의) ————— */
void Mecanum_MoveForward(Mecanum_t *m, float speed); // 전진
void Mecanum_MoveBackward(Mecanum_t *m, float speed); // 후진
void Mecanum_StrafeLeft(Mecanum_t *m, float speed); // 왼쪽 평행이동 (요청한 기능!)
void Mecanum_StrafeRight(Mecanum_t *m, float speed); // 오른쪽 평행이동
void Mecanum_TurnLeft(Mecanum_t *m, float speed); // 제자리 좌회전
```



```
void Mecanum_TurnRight(Mecanum_t *m, float speed);    // 제자리 우회전
#endif
```

mecanum.c

```
/**
 * @file   mecanum.c
 */
#include "mecanum.h"
#include <math.h>

static float abs_f(float x) { return x < 0 ? -x : x; }

void Mecanum_Init(Mecanum_t *m,
                  BTS7960_t *fl, BTS7960_t *fr,
                  BTS7960_t *rl, BTS7960_t *rr,
                  float max_output)
{
    m->wheels[WHEEL_FL] = fl;
    m->wheels[WHEEL_FR] = fr;
    m->wheels[WHEEL_RL] = rl;
    m->wheels[WHEEL_RR] = rr;

    if (max_output <= 0.0f) max_output = 0.5f;    // 안전 기본값
    if (max_output > 1.0f) max_output = 1.0f;
    m->max_output = max_output;

    Mecanum_Stop(m);
}

void Mecanum_Drive(Mecanum_t *m, float vx, float vy, float wz)
{
    /* 1) 입력 클램프 */
    if (vx > 1.0f) vx = 1.0f; if (vx < -1.0f) vx = -1.0f;
    if (vy > 1.0f) vy = 1.0f; if (vy < -1.0f) vy = -1.0f;
    if (wz > 1.0f) wz = 1.0f; if (wz < -1.0f) wz = -1.0f;

    /* 2) 메카넘 역운동학
     *
     *   FL = vx - vy - ω
     *   FR = vx + vy + ω
     *   RL = vx + vy - ω
     *   RR = vx - vy + ω
     */
    float w[WHEEL_COUNT];
    w[WHEEL_FL] = vx - vy - wz;
    w[WHEEL_FR] = vx + vy + wz;
    w[WHEEL_RL] = vx + vy - wz;
    w[WHEEL_RR] = vx - vy + wz;

    /* 3) 정규화 — 4개 중 최대 절댓값이 1.0 넘으면 모두 비례 축소
     *   (saturation 시 한쪽만 잘리면 차체가 의도와 다르게 움직임) */
    float max_mag = abs_f(w[0]);
    for (int i = 1; i < WHEEL_COUNT; i++) {
        float a = abs_f(w[i]);
        if (a > max_mag) max_mag = a;
    }
    if (max_mag > 1.0f) {
        for (int i = 0; i < WHEEL_COUNT; i++) w[i] /= max_mag;
    }

    /* 4) 안전 max_output 곱하고 모터 출력 */
    for (int i = 0; i < WHEEL_COUNT; i++) {
        BTS7960_SetSpeed(m->wheels[i], w[i] * m->max_output);
    }
}

void Mecanum_Stop(Mecanum_t *m)
{
    for (int i = 0; i < WHEEL_COUNT; i++) {
        BTS7960_Stop(m->wheels[i]);
    }
}

/* ————— 편의 동작 함수 ————— */
void Mecanum_MoveForward (Mecanum_t *m, float s) { Mecanum_Drive(m,  s, 0,  0); }
void Mecanum_MoveBackward(Mecanum_t *m, float s) { Mecanum_Drive(m, -s, 0,  0); }
void Mecanum_StrafeLeft  (Mecanum_t *m, float s) { Mecanum_Drive(m,  0,-s,  0); }
void Mecanum_StrafeRight (Mecanum_t *m, float s) { Mecanum_Drive(m,  0, s,  0); }
void Mecanum_TurnLeft    (Mecanum_t *m, float s) { Mecanum_Drive(m,  0, 0,  s); }
void Mecanum_TurnRight   (Mecanum_t *m, float s) { Mecanum_Drive(m,  0, 0, -s); }
```

A7. linesensor

linesensor.h

```
/**
 * @file    linesensor.h
 * @brief   TCRT5000 5채널 라인 센서 — 라인 위치 계산
 *
 * 5개 센서 배치 (앞쪽에서 봤을 때):
 *
 *  S1  S2  S3  S4  S5
 *  ───────────
 *  왼←      중앙      →오른
 *
 * 각 센서: 검은선 위 = LOW (0), 흰바닥 = HIGH (1)
 *          (TCRT5000 보드 종류에 따라 반대일 수도 → ACTIVE_LOW 매크로로 조정)
 *
 * 라인 위치 추정 (Weighted Average):
 * position =  $\Sigma(i \times b\_i) / \Sigma(b\_i)$ 
 * 여기서 b_i = i번째 센서가 검은선이면 1, 아니면 0
 *
 * position 범위: -2.0 ~ +2.0
 * -2 = 가장 왼쪽 (S1만 라인 검출)
 * 0 = 정중앙 (S3만 검출 또는 좌우 대칭)
 * +2 = 가장 오른쪽 (S5만 검출)
 *
 * 모든 센서가 흰바닥이면 (라인 없음) → 마지막 위치 유지 + lost 플래그 set
 */
#ifndef __LINESENSOR_H
#define __LINESENSOR_H

#include "stm32f4xx_hal.h"
#include <stdint.h>
#include <stdbool.h>

#define LINE_SENSOR_COUNT 5

/* 센서 출력 극성:
 * ACTIVE_LOW = 1 → 검은선 위에서 LOW (TCRT5000 일반 모듈 표준)
 * ACTIVE_LOW = 0 → 검은선 위에서 HIGH
 * 처음 컷을 때 부팅 메시지의 raw 값 보고 결정하면 됨
 */
#define LINE_ACTIVE_LOW 1

typedef struct {
    GPIO_TypeDef *port[LINE_SENSOR_COUNT];
    uint16_t pin [LINE_SENSOR_COUNT];

    uint8_t raw[LINE_SENSOR_COUNT]; // 0/1 비트 (전처리 후)
    float position; // -2.0 ~ +2.0
    bool line_lost; // 모든 센서가 흰바닥일 때 true
    float last_known_position; // lost 시 직전 값 유지
} LineSensor_t;

/**
 * @brief 5채널 라인 센서 초기화
 * port/pin 5쌍을 각각 인자로 받음
 */
void LineSensor_Init(LineSensor_t *ls,
                    GPIO_TypeDef *p1, uint16_t pin1,
                    GPIO_TypeDef *p2, uint16_t pin2,
                    GPIO_TypeDef *p3, uint16_t pin3,
                    GPIO_TypeDef *p4, uint16_t pin4,
                    GPIO_TypeDef *p5, uint16_t pin5);

/**
 * @brief 5채널 읽고 라인 위치 계산
 * 제어 루프마다 호출
 */
void LineSensor_Read(LineSensor_t *ls);

#endif
```

linesensor.c

```
/**
 * @file    linesensor.c
```

```

*/
#include "linesensor.h"

void LineSensor_Init(LineSensor_t *ls,
                    GPIO_TypeDef *p1, uint16_t pin1,
                    GPIO_TypeDef *p2, uint16_t pin2,
                    GPIO_TypeDef *p3, uint16_t pin3,
                    GPIO_TypeDef *p4, uint16_t pin4,
                    GPIO_TypeDef *p5, uint16_t pin5)
{
    ls->port[0] = p1; ls->pin[0] = pin1;
    ls->port[1] = p2; ls->pin[1] = pin2;
    ls->port[2] = p3; ls->pin[2] = pin3;
    ls->port[3] = p4; ls->pin[3] = pin4;
    ls->port[4] = p5; ls->pin[4] = pin5;

    for (int i = 0; i < LINE_SENSOR_COUNT; i++) ls->raw[i] = 0;
    ls->position = 0.0f;
    ls->line_lost = false;
    ls->last_known_position = 0.0f;
}

void LineSensor_Read(LineSensor_t *ls)
{
    /* 1) 5개 핀 읽기 + 극성 처리
     *   on_line = 1 이면 검은선 위 */
    int sum_b = 0; // 검은선 검출 센서 개수
    float weighted = 0.0f;

    /* 가중치: -2, -1, 0, +1, +2 (왼→오른) */
    static const float weight[LINE_SENSOR_COUNT] = {-2.0f, -1.0f, 0.0f, +1.0f, +2.0f};

    for (int i = 0; i < LINE_SENSOR_COUNT; i++) {
        GPIO_PinState s = HAL_GPIO_ReadPin(ls->port[i], ls->pin[i]);
        #if LINE_ACTIVE_LOW
            ls->raw[i] = (s == GPIO_PIN_RESET) ? 1 : 0; // LOW면 검은선
        #else
            ls->raw[i] = (s == GPIO_PIN_SET) ? 1 : 0;
        #endif
        if (ls->raw[i]) {
            weighted += weight[i];
            sum_b++;
        }
    }

    /* 2) 라인 위치 계산 */
    if (sum_b == 0) {
        /* 라인 없음 → 마지막 위치 유지하고 플래그 set */
        ls->line_lost = true;
        ls->position = ls->last_known_position;
    } else {
        ls->line_lost = false;
        ls->position = weighted / (float)sum_b;
        ls->last_known_position = ls->position;
    }
}

```

A8. linefollow

linefollow.h

```
/**
 * @file    linefollow.h
 * @brief   라인 추종 PID — 센서 위치 → 회전 명령
 *
 * 목표: 라인이 항상 중앙(position = 0)에 오도록 회전 보정
 *
 *  $error = 0 - position$  (라인이 오른쪽에 있으면  $error < 0 \rightarrow$  우회전)
 *  $\omega = K_p * e + K_i * \int e + K_d * de/dt$ 
 *
 * 본 시스템은 일반 PID 구조체 그대로 활용 가능하지만,
 * 라인트레이서 특성에 맞춘 thin wrapper 모듈 제공.
 *
 * 시연용 기본 동작:
 * - 라인 검출 정상 → 전진 + PID로 회전 보정
 * - 라인 잃음 → 정지 (또는 마지막 회전 방향으로 짧게 탐색)
 */
#ifndef __LINEFOLLOW_H
#define __LINEFOLLOW_H

#include "linesensor.h"
#include "pid.h"
#include "mecanum.h"

typedef struct {
    LineSensor_t *sensor;
    Mecanum_t *mecanum;
    PID_t pid; // 라인 위치 PID

    float base_speed; // 기본 전진 속도 (0.0~1.0)
    float turn_gain; // 회전 명령 스케일 (0.5~1.5 시작)
    bool stop_on_lost; // 라인 잃으면 정지 여부
} LineFollow_t;

/**
 * @brief 초기화
 * @param base_speed 기본 전진 속도 (0.2~0.4 부터 시작 추천)
 * @param Kp,Ki,Kd 라인 PID 게인 (예: 0.6, 0.0, 0.1)
 * @param dt_sec 제어 주기 (예: 0.01 = 100Hz)
 */
void LineFollow_Init(LineFollow_t *lf,
                    LineSensor_t *sensor, Mecanum_t *mecanum,
                    float base_speed,
                    float Kp, float Ki, float Kd,
                    float dt_sec);

/**
 * @brief 한 스텝 — 센서 읽고 PID 돌리고 메카넘에 명령
 * 제어 루프마다 호출
 */
void LineFollow_Update(LineFollow_t *lf);

/** @brief 비활성화 (정지) */
void LineFollow_Stop(LineFollow_t *lf);

#endif
```

linefollow.c

```
/**
 * @file    linefollow.c
 */
#include "linefollow.h"

void LineFollow_Init(LineFollow_t *lf,
                    LineSensor_t *sensor, Mecanum_t *mecanum,
                    float base_speed,
                    float Kp, float Ki, float Kd,
                    float dt_sec)
{
    lf->sensor = sensor;
    lf->mecanum = mecanum;
    lf->base_speed = base_speed;
    lf->turn_gain = 1.0f;
}
```

```

    lf->stop_on_lost = true;

    /* PID: 출력  $\pm 1.0$  (회전 명령은  $-1 \sim +1$ ), 적분 한계  $\pm 0.5$  */
    PID_Init(&lf->pid, Kp, Ki, Kd, -1.0f, 1.0f, 0.5f, dt_sec);
}

void LineFollow_Update(LineFollow_t *lf)
{
    /* 1) 센서 읽기 */
    LineSensor_Read(lf->sensor);

    /* 2) 라인 잃음 처리 */
    if (lf->sensor->line_lost) {
        if (lf->stop_on_lost) {
            Mecanum_Stop(lf->mecanum);
            return;
        }
        /* 또는: 직전 위치 부호로 천천히 그쪽으로 회전 탐색하는 로직 가능
        *      여기선 단순화하여 정지 */
    }

    /* 3) PID 계산
    *      setpoint = 0 (라인이 중앙에 와야 함)
    *      measurement = 라인 위치 (-2.0 ~ +2.0)
    *      출력 = 회전 명령  $\omega$  (양수 = 좌회전)
    *
    *      라인이 오른쪽(position > 0)에 있으면 우회전( $\omega < 0$ )으로 보정해야 함
    *      error = 0 - position = -position 이므로
    *      PID 출력이 그대로  $\omega$ 로 들어가면 부호가 맞음 (확인 필수)
    */
    float wz = PID_Compute(&lf->pid, 0.0f, lf->sensor->position);
    wz *= lf->turn_gain;

    /* 4) 메카닉에 명령 — 전진 + 회전 보정 */
    Mecanum_Drive(lf->mecanum, lf->base_speed, 0.0f, wz);
}

void LineFollow_Stop(LineFollow_t *lf)
{
    Mecanum_Stop(lf->mecanum);
    PID_Reset(&lf->pid);
}

```

A9. encoder

encoder.h

```
/**
 * @file    encoder.h
 * @brief   엔코더 EXTI 카운터 — 4채널
 *
 * JGB37-520 엔코더 2상 출력 (A상, B상) 중 A상만 EXTI 인터럽트로 받아 카운트.
 * 방향은 마지막 모터 PWM 명령 부호로 추정 (간이 방식).
 *
 * H/W Encoder Mode 안 쓰는 이유:
 * - F446RE Nucleo (LQFP64)에서 TIM2/3/4/5 Encoder Mode 4개를
 *   동시에 핀 매핑하기 어려움 (PWM 핀과 겹침)
 * - EXTI 4개면 핀 4개만 쓰기도 충분히 동작
 *
 * 정밀 PID가 필요하면 H/W Encoder Mode + B상까지 사용 권장.
 * 본 프로젝트는 시연용으로 펄스 카운트 + 누적 거리 추정 정도면 충분.
 *
 * 사용법 (CubeMX):
 * 1) 엔코더 A상 핀 4개를 GPIO_EXTI 로 설정 (Rising 또는 Both Edge)
 * 2) NVIC에서 해당 EXTI 라인 활성화
 * 3) main.c 에서 Encoder_OnInterrupt() 호출 (HAL_GPIO_EXTI_Callback)
 */
#ifndef __ENCODER_H
#define __ENCODER_H

#include "stm32f4xx_hal.h"
#include <stdint.h>

#define ENC_COUNT 4

typedef struct {
    volatile int32_t count[ENC_COUNT]; // 누적 카운트 (음수 가능)
    int8_t          direction[ENC_COUNT]; // +1 또는 -1, main 코드가 SetDirection 으로 갱신
    uint16_t         exti_pin[ENC_COUNT]; // 어느 EXTI 핀인지 (콜백에서 매칭용)
} Encoder_t;

/** @brief 초기화 (카운트 0, 방향 +1) */
void Encoder_Init(Encoder_t *e,
                  uint16_t pin_fl, uint16_t pin_fr,
                  uint16_t pin_rl, uint16_t pin_rr);

/**
 * @brief 모터 PWM 부호에 따라 방향 갱신
 * Mecanum_Drive 후 호출하는 게 좋음
 */
void Encoder_SetDirection(Encoder_t *e, int idx, float motor_speed);

/** @brief 카운트 읽기 (인터럽트 안전) */
int32_t Encoder_GetCount(Encoder_t *e, int idx);

/** @brief 모든 카운트 리셋 */
void Encoder_Reset(Encoder_t *e);

/**
 * @brief EXTI 콜백에서 호출 — GPIO_Pin 받아 어느 채널인지 매칭하고 카운트
 * main.c 의 HAL_GPIO_EXTI_Callback() 안에서 호출
 */
void Encoder_OnInterrupt(Encoder_t *e, uint16_t GPIO_Pin);

#endif
```

encoder.c

```
/**
 * @file    encoder.c
 */
#include "encoder.h"

void Encoder_Init(Encoder_t *e,
                  uint16_t pin_fl, uint16_t pin_fr,
                  uint16_t pin_rl, uint16_t pin_rr)
{
    e->exti_pin[0] = pin_fl;
    e->exti_pin[1] = pin_fr;
    e->exti_pin[2] = pin_rl;
```

```

e->exti_pin[3] = pin_rr;
for (int i = 0; i < ENC_COUNT; i++) {
    e->count[i] = 0;
    e->direction[i] = +1;
}
}

void Encoder_SetDirection(Encoder_t *e, int idx, float motor_speed)
{
    if (idx < 0 || idx >= ENC_COUNT) return;
    if (motor_speed > 0.0f) e->direction[idx] = +1;
    else if (motor_speed < 0.0f) e->direction[idx] = -1;
    /* speed == 0이면 직전 방향 유지 (관성으로 조금 더 굴러갈 수 있음) */
}

int32_t Encoder_GetCount(Encoder_t *e, int idx)
{
    if (idx < 0 || idx >= ENC_COUNT) return 0;
    /* int32_t 단일 변수 읽기는 ARM Cortex-M에서 atomic이라 별도 보호 불필요 */
    return e->count[idx];
}

void Encoder_Reset(Encoder_t *e)
{
    for (int i = 0; i < ENC_COUNT; i++) e->count[i] = 0;
}

void Encoder_OnInterrupt(Encoder_t *e, uint16_t GPIO_Pin)
{
    /* 4채널 중 일치하는 핀 찾기 */
    for (int i = 0; i < ENC_COUNT; i++) {
        if (GPIO_Pin == e->exti_pin[i]) {
            e->count[i] += e->direction[i];
            return;
        }
    }
}

```


A10. main.c USER CODE (통합본)

아래 코드는 CubeIDE가 자동 생성한 main.c 의 USER CODE 영역에 섹션별로 복붙하는 가이드입니다. 별도 파일이 아니라 main.c 안에 들어가는 내용입니다.

main_user_code.c

```
/*
 * STM32 Nucleo-F446RE
 * 2축 짐벌 + 메카닉 4륜 + 라인 트레이서 통합
 *
 * △ 이 파일은 CubeMX가 생성한 main.c의 USER CODE 영역에
 * 복붙하기 위한 가이드입니다. 별도 파일 아님!
 *
 * 동작:
 * TIM7 (200Hz, 5ms) → 짐벌 제어 (IMU + PID + 서보)
 * TIM6 (100Hz, 10ms) → 차량 제어 (라인 + 메카닉)
 * EXTI → 엔코더 펄스 카운트
 */

*
* 사용 타이머/주변장치 정리 (CubeMX에서 모두 설정 필요):
*
* I2C1 — MPU6050 (PB8, PB9) Fast 400kHz
* TIM1 CH1/2 — 짐벌 서보 PWM (PA8, PA9) 50Hz
* TIM8 CH1-4 — BTS7960 RPWM (PC6,PC7,PC8,PC9) 20kHz
* TIM4 CH1/2 + TIM5 CH3/4 — BTS7960 LPWM 20kHz
* TIM6 — 차량 100Hz 인터럽트 (NVIC 활성화)
* TIM7 — 짐벌 200Hz 인터럽트 (NVIC 활성화)
* EXTI — 엔코더 4채널 GPIO
* USART2 — 디버그 (PA2, PA3)
* GPIO 입력 5 — 라인 센서 (PB0~PB4 등)
*/

/*
 * USER CODE BEGIN Includes
 */
#include "mpu6050.h"
#include "attitude.h"
#include "pid.h"
#include "servo.h"
#include "bts7960.h"
#include "mecanum.h"
#include "linesensor.h"
#include "linefollow.h"
#include "encoder.h"
#include <stdio.h>
#include <string.h>
/* USER CODE END Includes */

/*
 * USER CODE BEGIN PD
 */

/* ————— 제어 주기 ————— */
#define GIMBAL_DT_SEC 0.005f // 200 Hz (TIM7)
#define VEHICLE_DT_SEC 0.010f // 100 Hz (TIM6)

/* ————— 짐벌 PID 게인 (튜닝 시작점) ————— */
#define PID_PITCH_KP 4.0f
#define PID_PITCH_KI 0.10f
#define PID_PITCH_KD 0.50f
#define PID_ROLL_KP 4.0f
#define PID_ROLL_KI 0.10f
#define PID_ROLL_KD 0.50f
#define PID_OUT_LIMIT 45.0f
#define PID_INT_LIMIT 20.0f

#define COMP_FILTER_ALPHA 0.98f

/* ————— 서보 보정 (조립 후 실측) ————— */
#define SERVO_PITCH_OFFSET 0.0f
#define SERVO_ROLL_OFFSET 0.0f
#define SERVO_PITCH_REVERSE 0
#define SERVO_ROLL_REVERSE 0

/* ————— 메카닉 PWM 타이머 ARR —————
```

```

* 20kHz PWM → 180MHz / (PSC+1) / (ARR+1) = 20000
* PSC = 0, ARR = 8999 → 분해능 9000 단계
*/
#define MOTOR_PWM_ARR 8999

/* ----- 메카넘 안전 최대 출력 -----
* 처음에 작게! 큰 차체가 갑자기 튀어나가면 위험.
*/
#define MECANUM_MAX_OUT 0.4f

/* ----- 라인 추종 PID ----- */
#define LINE_BASE_SPEED 0.3f
#define LINE_KP 0.6f
#define LINE_KI 0.0f
#define LINE_KD 0.10f

/* ----- 모터 방향 반전 (조립 후 단독 테스트로 결정) ----- */
#define WHEEL_FL_REVERSE 0
#define WHEEL_FR_REVERSE 0
#define WHEEL_RL_REVERSE 0
#define WHEEL_RR_REVERSE 0

#define DEBUG_PRINT_DECIM 50 // 100Hz * 50 = 0.5s

/* ----- 시스템 모드 ----- */
typedef enum {
    MODE_IDLE = 0, // 모든 동작 정지
    MODE_GIMBAL_ONLY, // 짐벌만 동작 (튜닝/테스트)
    MODE_MANUAL_DRIVE, // UART 명령으로 차량 + 짐벌
    MODE_LINE_FOLLOW, // 라인 추종 + 짐벌 ← 발표 시연
} SystemMode_t;

/* USER CODE END PD */

/* -----
* USER CODE BEGIN PV
* ----- */
extern I2C_HandleTypeDef hi2c1;
extern TIM_HandleTypeDef htim1; // 짐벌 서보 PWM
extern TIM_HandleTypeDef htim4; // 메카넘 LPWM
extern TIM_HandleTypeDef htim5; // 메카넘 LPWM
extern TIM_HandleTypeDef htim6; // 차량 100Hz 인터럽트
extern TIM_HandleTypeDef htim7; // 짐벌 200Hz 인터럽트
extern TIM_HandleTypeDef htim8; // 메카넘 RPWM
extern UART_HandleTypeDef huart2;

/* 짐벌 */
MPU6050_t mpu;
Attitude_t attitude;
PID_t pid_pitch, pid_roll;
Servo_t servo_pitch, servo_roll;

/* 메카넘 */
BTS7960_t wheel_fl, wheel_fr, wheel_rl, wheel_rr;
Mecanum_t mecanum;

/* 라인 */
LineSensor_t line_sensor;
LineFollow_t line_follow;

/* 엔코더 */
Encoder_t encoder;

/* 시스템 상태 */
volatile SystemMode_t system_mode = MODE_IDLE;
volatile uint8_t gimbal_flag = 0;
volatile uint8_t vehicle_flag = 0;
volatile uint32_t debug_cnt = 0;

volatile uint8_t uart_rx_byte = 0;
volatile uint8_t uart_cmd_ready = 0;

/* USER CODE END PV */

/* -----
* USER CODE BEGIN PFP
* ----- */
static void System_Init(void);
static void Gimbal_ControlLoop(void);
static void Vehicle_ControlLoop(void);

```

```

static void Debug_Print(void);
static void Error_Trap(const char *msg);
static void Process_UartCommand(void);
/* USER CODE END PFP */

/*
 * USER CODE BEGIN 0 — printf retarget + UART RX 콜백
 */
#ifdef __GNUC__
int __io_putchar(int ch)
{
    HAL_UART_Transmit(&huart2, (uint8_t*)&ch, 1, 10);
    return ch;
}
#endif

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2) {
        uart_cmd_ready = 1;
        HAL_UART_Receive_IT(&huart2, (uint8_t*)&uart_rx_byte, 1);
    }
}
/* USER CODE END 0 */

/*
 * USER CODE BEGIN 2 — HAL_Init 후, while(1) 직전
 */
System_Init();

HAL_UART_Receive_IT(&huart2, (uint8_t*)&uart_rx_byte, 1);

HAL_TIM_Base_Start_IT(&tim7); // 짐벌 200Hz
HAL_TIM_Base_Start_IT(&tim6); // 차량 100Hz

printf("\r\nReady. Commands:\r\n");
printf(" '1' = Gimbal only\r\n");
printf(" '2' = Manual drive + Gimbal\r\n");
printf(" '3' = Line follow + Gimbal (DEMO)\r\n");
printf(" '0' = STOP all\r\n");
printf(" w/a/s/d/q/e = forward/strafeL/back/strafeR/turnL/turnR (mode 2)\r\n");
printf(" space      = stop wheels\r\n");

system_mode = MODE_GIMBAL_ONLY;

/* USER CODE END 2 */

/*
 * USER CODE BEGIN WHILE — main loop
 */
while (1)
{
    if (uart_cmd_ready) {
        uart_cmd_ready = 0;
        Process_UartCommand();
    }

    if (gimbal_flag) {
        gimbal_flag = 0;
        Gimbal_ControlLoop();
    }

    if (vehicle_flag) {
        vehicle_flag = 0;
        Vehicle_ControlLoop();

        if (++debug_cnt >= DEBUG_PRINT_DECIM) {
            debug_cnt = 0;
            Debug_Print();
        }
    }
}
/* USER CODE END WHILE */
/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */

/*
 * USER CODE BEGIN 4

```

```

static void System_Init(void)
{
    printf("\r\n===== \r\n");
    printf("  Gimbal + Mecanum Line Tracer Boot\r\n");
    printf("===== \r\n");

    /* 짐벌 */
    if (MPU6050_Init(&hi2c1) != HAL_OK) Error_Trap("MPU6050 init");
    printf("[OK] MPU6050\r\n");

    printf("Calibrating gyro... DO NOT MOVE!\r\n");
    HAL_Delay(800);
    if (MPU6050_CalibrateGyro(&hi2c1, &mpu) != HAL_OK) Error_Trap("Gyro cal");
    printf("[OK] Gyro bias\r\n");

    MPU6050_ReadAll(&hi2c1, &mpu);
    Attitude_Init(&attitude, &mpu, COMP_FILTER_ALPHA, GIMBAL_DT_SEC);

    PID_Init(&pid_pitch, PID_PITCH_KP, PID_PITCH_KI, PID_PITCH_KD,
            -PID_OUT_LIMIT, PID_OUT_LIMIT, PID_INT_LIMIT, GIMBAL_DT_SEC);
    PID_Init(&pid_roll,  PID_ROLL_KP,  PID_ROLL_KI,  PID_ROLL_KD,
            -PID_OUT_LIMIT, PID_OUT_LIMIT, PID_INT_LIMIT, GIMBAL_DT_SEC);

    Servo_Init(&servo_pitch, &htim1, TIM_CHANNEL_1,
              SERVO_PITCH_OFFSET, SERVO_PITCH_REVERSE);
    Servo_Init(&servo_roll,  &htim1, TIM_CHANNEL_2,
              SERVO_ROLL_OFFSET,  SERVO_ROLL_REVERSE);
    Servo_StartAll(&htim1);
    Servo_WriteAngle(&servo_pitch, 0.0f);
    Servo_WriteAngle(&servo_roll,  0.0f);
    printf("[OK] Gimbal\r\n");

    /* 메카넘 4륜 — 편은 CubeMX와 일치시킬 것 */
    BTS7960_Init(&wheel_fl, &htim8, TIM_CHANNEL_1, &htim4, TIM_CHANNEL_1,
                MOTOR_PWM_ARR, WHEEL_FL_REVERSE);
    BTS7960_Init(&wheel_fr, &htim8, TIM_CHANNEL_2, &htim4, TIM_CHANNEL_2,
                MOTOR_PWM_ARR, WHEEL_FR_REVERSE);
    BTS7960_Init(&wheel_rl, &htim8, TIM_CHANNEL_3, &htim5, TIM_CHANNEL_3,
                MOTOR_PWM_ARR, WHEEL_RL_REVERSE);
    BTS7960_Init(&wheel_rr, &htim8, TIM_CHANNEL_4, &htim5, TIM_CHANNEL_4,
                MOTOR_PWM_ARR, WHEEL_RR_REVERSE);

    Mecanum_Init(&mecanum, &wheel_fl, &wheel_fr, &wheel_rl, &wheel_rr,
                MECANUM_MAX_OUT);
    printf("[OK] Mecanum\r\n");

    /* 라인 센서 — 편 5개는 CubeMX에서 GPIO_Input 설정 */
    LineSensor_Init(&line_sensor,
                  GPIOB, GPIO_PIN_0,
                  GPIOB, GPIO_PIN_1,
                  GPIOB, GPIO_PIN_2,
                  GPIOB, GPIO_PIN_3,
                  GPIOB, GPIO_PIN_4);

    LineFollow_Init(&line_follow, &line_sensor, &mecanum,
                  LINE_BASE_SPEED, LINE_KP, LINE_KI, LINE_KD,
                  VEHICLE_DT_SEC);
    printf("[OK] Line\r\n");

    /* 엔코더 EXTI */
    Encoder_Init(&encoder,
                GPIO_PIN_0, GPIO_PIN_1, GPIO_PIN_4, GPIO_PIN_15);
    printf("[OK] Encoder\r\n");

    HAL_Delay(500);
    printf("All systems GO.\r\n");
}

static void Gimbal_ControlLoop(void)
{
    if (MPU6050_ReadAll(&hi2c1, &mpu) != HAL_OK) return;
    Attitude_Update(&attitude, &mpu);

    if (system_mode == MODE_IDLE) {
        Servo_WriteAngle(&servo_pitch, 0.0f);
        Servo_WriteAngle(&servo_roll,  0.0f);
        return;
    }

    float u_pitch = PID_Compute(&pid_pitch, 0.0f, attitude.pitch);

```

```

float u_roll = PID_Compute(&pid_roll, 0.0f, attitude.roll);

Servo_WriteAngle(&servo_pitch, u_pitch);
Servo_WriteAngle(&servo_roll, u_roll);
}

static void Vehicle_ControlLoop(void)
{
    switch (system_mode) {
        case MODE_IDLE:
        case MODE_GIMBAL_ONLY:
            Mecanum_Stop(&mecanum);
            break;

        case MODE_MANUAL_DRIVE:
            /* UART 명령에서 Mecanum_Drive 호출 — 여기서는 유지만 */
            break;

        case MODE_LINE_FOLLOW:
            LineFollow_Update(&line_follow);
            break;
    }

    /* 엔코더 방향 갱신 — 현재 PWM 부호로 추정 */
    int32_t r, l;

    r = __HAL_TIM_GET_COMPARE(wheel_fl.htim_rpwm, wheel_fl.ch_rpwm);
    l = __HAL_TIM_GET_COMPARE(wheel_fl.htim_lpwm, wheel_fl.ch_lpwm);
    Encoder_SetDirection(&encoder, 0, (float)(r - l));

    r = __HAL_TIM_GET_COMPARE(wheel_fr.htim_rpwm, wheel_fr.ch_rpwm);
    l = __HAL_TIM_GET_COMPARE(wheel_fr.htim_lpwm, wheel_fr.ch_lpwm);
    Encoder_SetDirection(&encoder, 1, (float)(r - l));

    r = __HAL_TIM_GET_COMPARE(wheel_rl.htim_rpwm, wheel_rl.ch_rpwm);
    l = __HAL_TIM_GET_COMPARE(wheel_rl.htim_lpwm, wheel_rl.ch_lpwm);
    Encoder_SetDirection(&encoder, 2, (float)(r - l));

    r = __HAL_TIM_GET_COMPARE(wheel_rr.htim_rpwm, wheel_rr.ch_rpwm);
    l = __HAL_TIM_GET_COMPARE(wheel_rr.htim_lpwm, wheel_rr.ch_lpwm);
    Encoder_SetDirection(&encoder, 3, (float)(r - l));
}

static void Process_UartCommand(void)
{
    uint8_t c = uart_rx_byte;

    switch (c) {
        case '0': system_mode = MODE_IDLE;          printf(">> IDLE\r\n");          return;
        case '1': system_mode = MODE_GIMBAL_ONLY;    printf(">> Gimbal only\r\n");    return;
        case '2': system_mode = MODE_MANUAL_DRIVE;    printf(">> Manual\r\n");        return;
        case '3': system_mode = MODE_LINE_FOLLOW;
            LineFollow_Stop(&line_follow);
            printf(">> Line follow\r\n"); return;
        case 'r': Encoder_Reset(&encoder); printf(">> Enc reset\r\n"); return;
    }

    if (system_mode != MODE_MANUAL_DRIVE) return;

    switch (c) {
        case 'w': Mecanum_MoveForward (&mecanum, 0.4f); printf("> FWD\r\n"); break;
        case 's': Mecanum_MoveBackward(&mecanum, 0.4f); printf("> BWD\r\n"); break;
        case 'a': Mecanum_StrafeLeft (&mecanum, 0.4f); printf("> LEFT(strafe)\r\n"); break;
        case 'd': Mecanum_StrafeRight (&mecanum, 0.4f); printf("> RIGHT(strafe)\r\n"); break;
        case 'q': Mecanum_TurnLeft (&mecanum, 0.4f); printf("> TURN L\r\n"); break;
        case 'e': Mecanum_TurnRight (&mecanum, 0.4f); printf("> TURN R\r\n"); break;
        case ' ': Mecanum_Stop(&mecanum);          printf("> STOP\r\n"); break;
    }
}

static void Debug_Print(void)
{
    static const char *mode_name[] = { "IDLE", "GIMB", "MAN ", "LINE" };
    printf("[%s] P:%+6.2f R:%+6.2f | Line:%+5.2f%s | Enc[%5ld %5ld %5ld %5ld]\r\n",
        mode_name[system_mode],
        attitude.pitch, attitude.roll,
        line_sensor.position, line_sensor.line_lost ? "(LOST)" : " ",
        (long)Encoder_GetCount(&encoder, 0),
        (long)Encoder_GetCount(&encoder, 1),
    );
}

```

```

        (long)Encoder_GetCount(&encoder, 2),
        (long)Encoder_GetCount(&encoder, 3));
}

static void Error_Trap(const char *msg)
{
    printf("\r\nFATAL: %s\r\n", msg);
    Mecanum_Stop(&mecanum);
    Servo_WriteAngle(&servo_pitch, 0.0f);
    Servo_WriteAngle(&servo_roll, 0.0f);
    while (1) {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        HAL_Delay(100);
    }
}

/* ----- HAL 콜백 ----- */

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM7) gimbal_flag = 1;
    else if (htim->Instance == TIM6) vehicle_flag = 1;
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    Encoder_OnInterrupt(&encoder, GPIO_Pin);
}

/* USER CODE END 4 */

```

끝.

한 단계씩 검증하며 가세요. 일주일이면 짐벌, 2주면 차량, 3주면 통합, 4주면 발표까지
가능합니다.

막히면 어느 단계인지, 어떤 에러인지 명확히 적어 물어보세요.

화이팅!